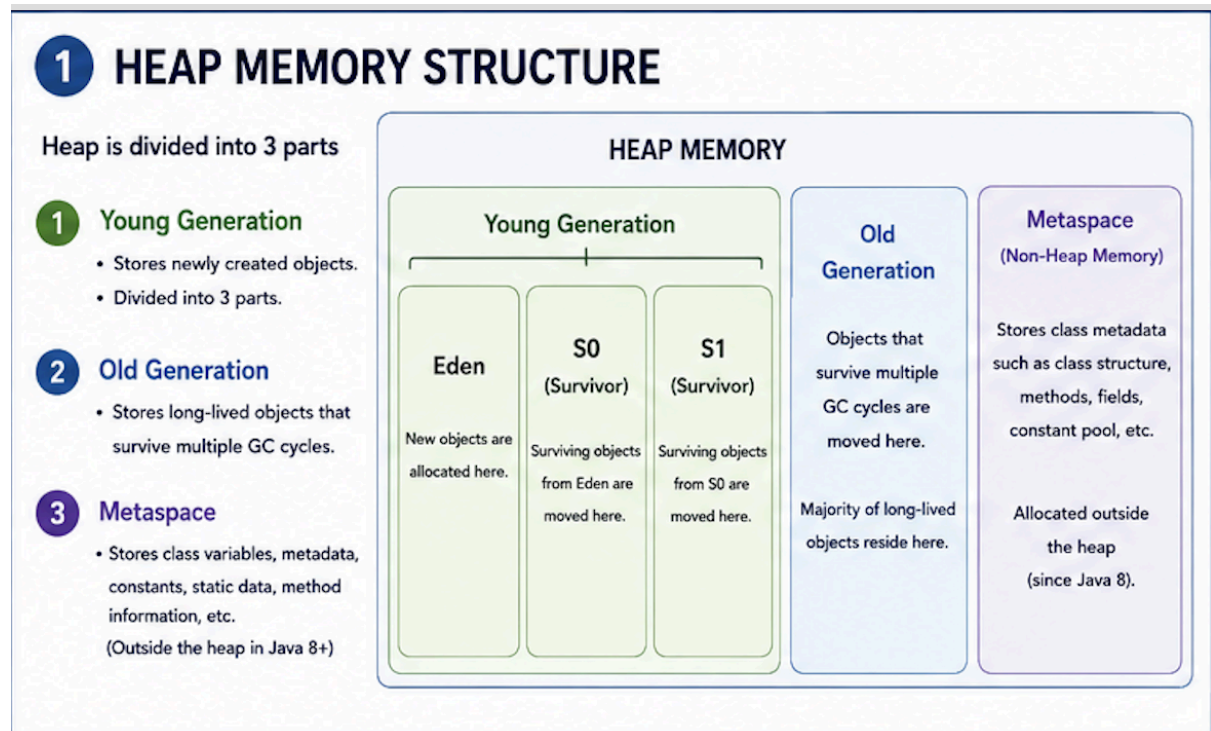


Java Heap Memory & Garbage Collection

Heap Memory



Heap memory is mainly divided into 3 parts:

1. Young Generation
2. Old Generation (Tenured Generation)
3. Metaspace

Young Generation

Young Generation is further divided into 3 parts:

1. Eden Space

- Whenever a new object is created, it is first allocated in Eden Space.

2. Survivor 0 (S0)

3. Survivor 1 (S1)

The Survivor spaces store objects that survive garbage collection.

Object Creation Flow

When we create an object:

Object → Eden

After some time, Garbage Collection (GC) is invoked.

Mark Phase

GC identifies objects that are no longer reachable (objects having no references).

Example:

```
Employee e = new Employee();  
e = null;
```

The Employee object becomes unreachable and is marked for deletion.

Sweep Phase

The GC removes unreachable objects and keeps the reachable objects.

Surviving objects are moved:

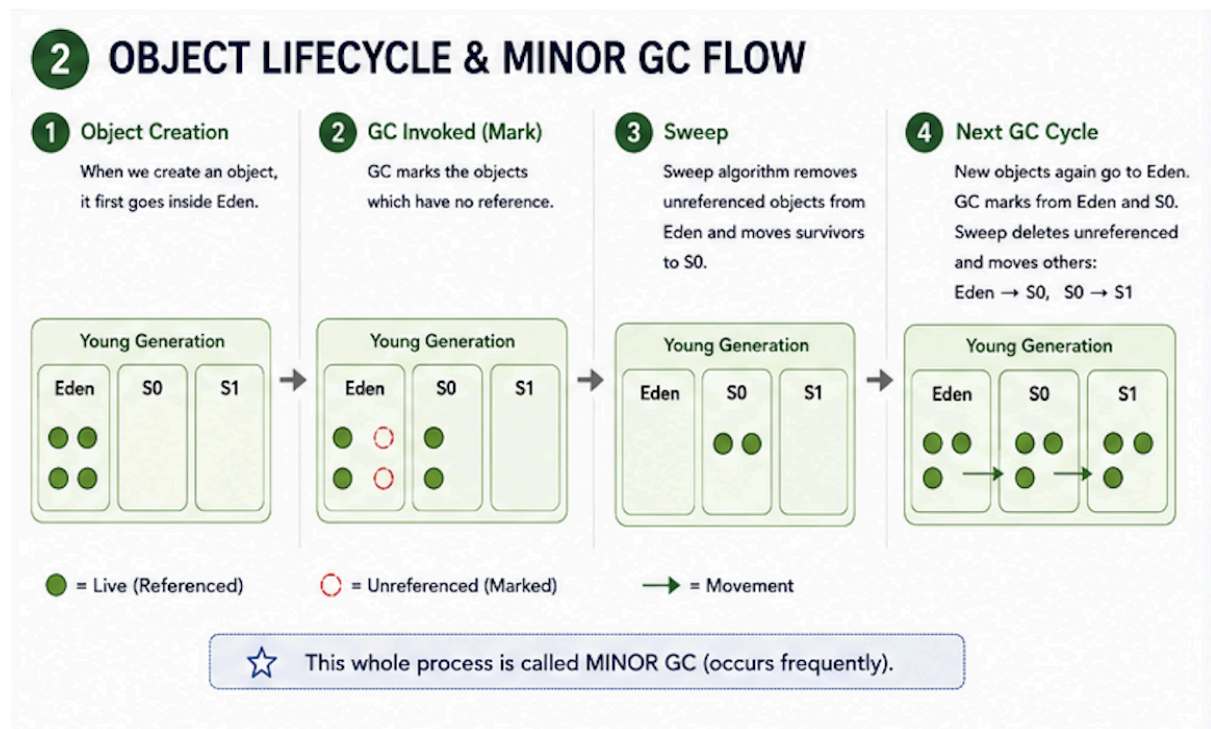
Eden → S0

Later:

S0 → S1

This process is known as Minor GC (Young GC).

Minor GC Flow



Cycle 1

Eden



S0 (Age = 1)

Cycle 2

Eden



S0



S1 (Age = 2)

Cycle 3

Eden



S0



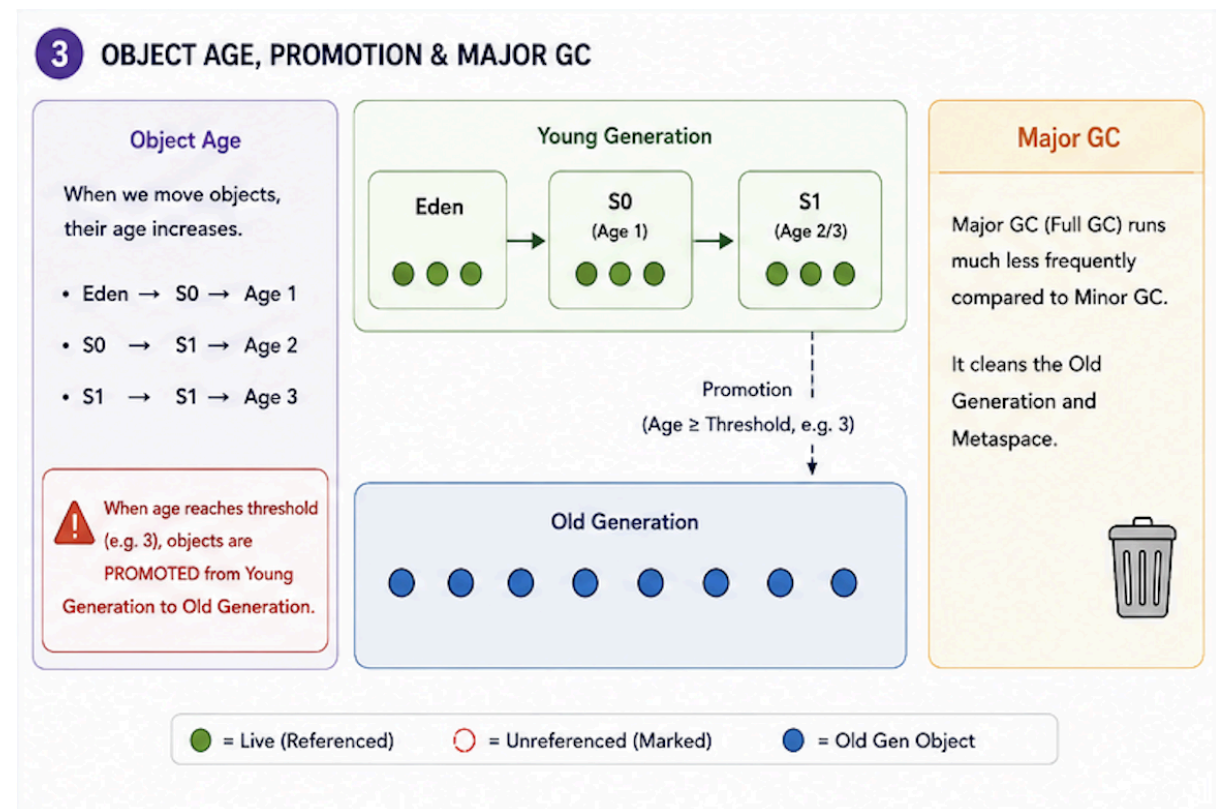
S1 (Age = 3)

Each time an object survives a Minor GC, its age increases.

Note:

In reality S0 and S1 keep swapping roles after every Minor GC.

Object Promotion



When an object's age reaches a threshold value, it is promoted to the Old Generation.

Example:

Age ≥ Threshold (commonly 15 in HotSpot JVM)

Young Generation

↓

Old Generation

Note:

You mentioned threshold 3. That is useful for understanding the concept, but in most JVMs the default MaxTenuringThreshold is 15.

Old Generation

Objects that survive multiple Minor GCs are moved to the Old Generation.

Examples:

- Long-lived objects
- Application caches
- Singleton objects

Major GC / Full GC works mainly on the Old Generation.

Major GC does not run as frequently as Minor GC because:

- Most objects die young.
 - Cleaning Old Generation is more expensive.
-

Metaspace

Metaspace stores:

- Class metadata
- Method metadata
- Runtime Constant Pool
- Static variables
- Class definitions
- JVM internal structures

Important:

Before Java 8:

- Permanent Generation (PermGen)

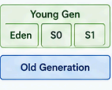
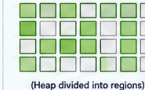
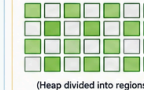
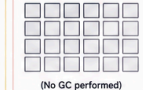
Java 8 onwards:

- Metaspace
- Uses native memory instead of heap memory.

Note:

Static variables logically belong to classes and their metadata is managed through Metaspace, although the actual object instances referenced by static variables still reside in Heap.

Garbage Collectors

JAVA GARBAGE COLLECTORS – OVERVIEW							
	1. Serial GC (Client Mode)	2. Parallel GC (Throughput Collector)	3. CMS GC (Concurrent Mark Sweep)	4. G1 GC (Garbage First)	5. ZGC (Z Garbage Collector)	6. Shenandoah GC (Low Pause Collector)	7. Epsilon GC (No-Op GC)
	 Single threaded collector	 Multi-threaded collector	 Concurrent collector (Old Gen focused)	 Region based, balanced collector	 Ultra low pause time collector	 Low pause time collector	 No garbage collection (Do nothing)
Description	1 GC Thread	Multiple GC Threads (Depends on CPU cores)	Multiple GC Threads	Multiple GC Threads	Multiple GC Threads	Multiple GC Threads	No GC Thread
STW (Stop The World)	Yes (Full pause)	Yes (Full pause)	Yes (But less frequent and shorter pauses)	Yes (But very short pauses)	Yes (Extremely short, almost negligible)	Yes (Short pauses)	No
Parallelism	No	Yes (During GC)	Yes (Most phases run concurrently)	Yes (Most phases run concurrently)	Yes (Concurrent)	Yes (Concurrent)	N/A
Compaction	Yes	Yes	No	Yes	Yes	Yes	No
Best For	Small applications, low heap size, single CPU machines	High throughput applications, batch jobs	Applications needing low pause times (Java 8 and below)	Large heap applications, balances throughput and pause time	Very large heap, low latency applications	Large applications with low pause time requirements	Performance testing, benchmarks, short-lived apps
Throughput	☆☆☆☆☆ (Low)	★★★★☆ (High)	★★★★☆ (Medium)	★★★★☆ (High)	★★★★☆ (Very High)	★★★★☆ (High)	★★★★★ (Highest)
Pause Time	☆☆☆☆☆ (High)	★★★★☆ (High)	★★★★☆ (Medium to Low)	★★★★☆ (Low)	★★★★★ (Very Low)	★★★★☆ (Low)	★★★★★ (No pause)
Heap Layout							
Available Since	JDK 1.2	JDK 1.4	JDK 1.5 (Deprecated in JDK 9, Removed in JDK 14)	JDK 7 (Experimental) JDK 9+ (Default GC)	JDK 11 (Experimental) JDK 15+ (Production Ready)	JDK 12 (Experimental) JDK 15+ (Production Ready)	JDK 11
 Notes: • Young Gen = Eden + S0 + S1 (Survivor Spaces) • Old Gen = Tenured Generation • STW = Application threads are paused  Region (In Use)  Region (Free)  Region (Set of Objects)  GC Selection (General Guidance) • Small / Simple App → Serial GC • High Throughput → Parallel GC • Low Pause (Java 8 and below) → CMS GC • Balanced (Default, General) → G1 GC • Very Low Pause, Huge Heap → ZGC / Shenandoah • Benchmark / Testing → Epsilon GC							

Java Garbage Collectors

1. Serial GC

Uses only one GC thread.

Advantages

- Simple
- Low memory overhead

Disadvantages

- Very slow for large applications.
- Application threads stop during GC.

This is called:

Stop-The-World (STW)

Flow

Application Threads

Pause

↓

GC Thread Runs

↓

Application Resumes

Best suited for

- Small applications
- Single CPU environments

JVM Option

-XX:+UseSerialGC

2. Parallel GC

Uses multiple GC threads.

Number of GC threads generally depends on available CPU cores.

Advantages

- Faster than Serial GC
- Better throughput

Disadvantages

- Application threads still pause during GC.
- Stop-The-World still happens.

Best suited for

- High throughput applications
- Batch processing systems

JVM Option

-XX:+UseParallelGC

3. CMS (Concurrent Mark Sweep)

CMS = Concurrent Mark Sweep

Works concurrently with application threads.

Advantages

- Lower pause times
- Better responsiveness

Disadvantages

- Not 100% pause-free.
- Memory fragmentation can occur.
- No memory compaction.

Because no compaction occurs, free memory may become fragmented.

CMS has been deprecated and removed in modern JVMs.

JVM Option

-XX:+UseConcMarkSweepGC

4. G1 GC (Garbage First)

Default GC in modern JVMs.

Instead of Young and Old being continuous memory blocks, G1 divides heap into many small regions.

Advantages

- Very low pause times
- Concurrent execution
- Memory compaction supported
- Better for large heaps
- Predictable GC behavior

G1 tries to meet pause-time goals while maximizing throughput.

Important

G1 does NOT provide a 100% guarantee that application threads never stop.

There are still some Stop-The-World phases, but pause times are usually much smaller than previous collectors.

Best suited for

- Large applications
- General purpose workloads
- Most enterprise Java applications

JVM Option

-XX:+UseG1GC

5. ZGC (Z Garbage Collector)

ZGC is designed for extremely low pause times even with very large heap sizes.

It performs most of its work concurrently with application threads.

Advantages

- Pause times usually less than 1 ms
- Supports very large heaps (TBs)
- Highly scalable
- Concurrent compaction

Disadvantages

- Slightly lower throughput than Parallel GC
- More complex internally

Best suited for

- Real-time systems
- Trading systems
- Large memory applications
- Applications requiring ultra-low latency

JVM Option

-XX:+UseZGC

Interview Point

ZGC focuses on **low latency** rather than maximum throughput.

6. Shenandoah GC

Developed by Red Hat.

Shenandoah performs garbage collection concurrently with application threads.

Advantages

- Very low pause times
- Concurrent compaction
- Suitable for large heaps

Disadvantages

- Slight throughput overhead
- Not available in some older JVM distributions

Best suited for

- Low-latency applications
- Large heap applications

JVM Option

-XX:+UseShenandoahGC

Interview Point

Shenandoah's goal is to make pause time independent of heap size.

7. Epsilon GC

Epsilon is known as the "No-Op Garbage Collector".

It does not perform any garbage collection.

Objects continue to accumulate until memory is exhausted.

Advantages

- Zero GC overhead

- Useful for performance testing

Disadvantages

- Heap eventually becomes full
- Application crashes with OutOfMemoryError

Best suited for

- Benchmarking
- Performance testing
- Short-lived applications

JVM Option

-XX:+UseEpsilonGC

Interview Point

Epsilon GC intentionally performs no garbage collection.

Quick Comparison

GC	Threads	Pause Time	Compaction	Status
Serial	Single	High	Yes	Active
Parallel	Multiple	High	Yes	Active
CMS	Multiple	Medium	No	Removed
G1	Multiple	Low	Yes	Default
ZGC	Multiple	Very Low (<1ms)	Yes	Active
Shenandoah	Multiple	Very Low	Yes	Active
Epsilon	None	None	No GC	Active

Interview Answer

How many Garbage Collectors are available in Java?

1. Serial GC
2. Parallel GC
3. CMS GC
4. G1 GC
5. ZGC
6. Shenandoah GC
7. Epsilon GC

Modern JVMs primarily use:

- G1 GC
- ZGC
- Shenandoah GC

Legacy collectors:

- Serial GC
- Parallel GC
- CMS GC

Special-purpose collector:

- Epsilon GC
-

Interview Questions

Q1. Difference between Minor GC and Major GC?

Minor GC:

- Works on Young Generation.
- Runs frequently.
- Faster.

Major GC:

- Works on Old Generation.
 - Runs less frequently.
 - More expensive.
-

Q2. Where are new objects created?

Answer:
Eden Space.

Q3. What is object promotion?

Answer:
Moving a surviving object from Young Generation to Old Generation after it reaches the age threshold.

Q4. Why are there two Survivor spaces?

Answer:
To efficiently copy surviving objects between survivor areas during Minor GC and avoid memory fragmentation.

Q5. Which GC is default in modern JVMs?

Answer:
G1 Garbage Collector.